Basketball RAG

NLP Course Project

Saad Imam 27079

Github Repository Link
App Link (Hosted on Hugging face)

1 Business Problem

Basketball is a sport whose technical depth is underestimated. At the casual level, fans can follow a game but struggle to understand *why* a referee call was made, how a specific play exploits a defensive scheme, or what the structural difference is between a pick-and-roll and a pick-and-pop. At the coaching level, communicating precise rules interpretations to players, or explaining the principles behind a tactical set, requires access to dense, technically heavy rulebooks and strategy literature.

The core problem this system addresses is the **accessibility gap** between institutional basketball knowledge (official rulebooks, tactical encyclopedias) and the people who need that knowledge most. Existing tools either expose raw PDFs with no semantic search, or require a domain expert to answer every question.

The inspiration for the system's framing came from *Mind the Game* and the older *Old Man and the Three* podcasts hosted by JJ Redick. As a basketball fan who started watching the game in 2020, I had issues with understanding what was happening on the court tactically. That is when I came across JJ Redick, a 15 year NBA veteran, who had 15-20 mins segments in his podcast explaining specific plays in clear, understandable language. This system aims to replicate that quality: answering questions from a casual fan asking "what is a defensive three-second violation?" and a coach asking "how does a Princeton offense attack a zone defence?" with equal precision.

The system deliberately excludes statistical lookup (e.g., "who led the league in three-point percentage in 2021?"), as this requires structured databases and is a qualitatively different problem from rules and strategy retrieval.

1.1 Expected Challenges

Several challenges were anticipated at the outset:

Corpus differences: The two data layers (official rulebooks in PDF format and a scraped tactical encyclopedia) differ significantly in structure, tone, vocabulary, and document length. A unified retrieval strategy must serve both equally well.

Technical query specificity: Queries about basketball rules often contain precise identifiers — rule numbers, section titles, league names — that a purely semantic dense retriever may not surface correctly. Hybrid retrieval was therefore identified as a core requirement from the beginning.

Faithfulness vs. fluency trade-off: A language model with parametric basketball knowledge may confidently answer a question without grounding its answer in the retrieved context. The system must enforce strict grounding to be trustworthy, at some cost to response fluency.

2 Platform Details

The system is deployed as a **Docker Space on Hugging Face Spaces**, exposing a **FastAPI** backend on port 7860 with a coded single-page front-end. Source code is hosted on GitHub, with a GitHub Actions workflow (`.github/workflows/sync-to-hf.yml`) that automatically pushes to the Hugging Face Space repository on every push to the `main` branch, keeping the two repositories in sync without manual input.

Some Key Notes:

1. **Hugging Face Spaces (Docker SDK):** Provides free-tier hosted inference without requiring a dedicated server. The Docker SDK was chosen over Gradio or Streamlit to avoid the constraints those frameworks impose on UI design; it was a personal goal to have a customized front-end rather than looking generic.

2. **Local Machine vs. Kaggle:** For the scraping and chunking tasks, the local machine was used as it was not a very GPU intensive task. However, BGE-M3 embedding of the full corpus was computationally infeasible on a local CPU within a reasonable time window. A Kaggle notebook with dual T4 GPUs was used to run the embedding and upsert pipeline, with embeddings saved to a local checkpoint file before upserting to Pinecone, so that a failed upsert does not require re-running the 30–60 minute embedding step.

3. The corpus files (raw PDFs and scraped JSON) are **not** included in the Hugging Face repository; all vectors and metadata reside in Pinecone and are queried at inference time. Model artefacts (BGE-M3, CrossEncoder) are downloaded lazily on the first request to avoid blocking the `uvicorn` server from binding to port 7860 during startup — an issue that had previously caused the Space to timeout and loop indefinitely.

3 Corpus Details

The corpus is organised into two layers:

3.1 Layer 1

- **NBA Official Playing Rules (2025–26)** This document covers all NBA rules across numbered rules and roman-numeral sections (e.g., Rule 10, Section XIII — Traveling), used for officiating games.
- **FIBA Official Basketball Rules 2024 (v10a).** Retrieved from the FIBA official website. Covers international rules, including differences in shot clock duration, foul limits, and game structure relative to the NBA.

Several page ranges were excluded from processing because they carry no retrieval value: for the NBA rulebook, pages 1–8 (the rules index and court diagram) and pages 72–76 (referee hand signals, which are image-based); for the FIBA rulebook, the front matter and appendix pages containing diagrams and administrative content. These exclusions were implemented by checking each chunk's page structure; more on this later.

Diagrams (X-and-O illustrations) present in both rulebooks were intentionally excluded. Current vision-language models do not reliably interpret the symbolic notation used in basketball diagrams, and including OCR artefacts from these pages would introduce noise rather than signal. Was also out of the scope of the project.

3.2 Layer 2

Layer 2 was scraped from `https://hoopstudent.com`, a basketball strategy and glossary website with three main category pages: *Basketball Glossary*, *Offense*, and *Defense*. Each category page lists terms with a 3–4 line concise definition and a link to a dedicated detail page. The scraper (`scraper.py`) performs the following steps for each term:

1. Parses the category hub page to extract the term name, concise definition, and detail URL.
2. Fetches the detail page and parses its content into structured sections, using heading elements (`h2`/`h3`) as section boundaries.
3. Structures the output as a JSON document with fields: `doc_id`, `term`, `category`, `source_url`, `layer`, `concise_definition`, and `sections[]`.

The decision to use JSON rather than plain text was made deliberately: structured metadata storage is essential for filtering and re-ranking during retrieval. With flat text files, information such as the source layer, term name, and category is either embedded in the text or lost entirely. With JSON, this information is preserved separately and stored alongside the vector in Pinecone.

The corpus does not include raw statistics, CSV box scores, or structured player records. The system is scoped to rules, definitions, and strategy — a deliberate framing that keeps retrieval tractable and the answer quality verifiable.

The directory structure of the processed corpus is as follows:

```
corpus/
  raw/
    Official-2025-26-NBA-Playing-Rules.pdf
    fiba-official-rules-2024-v10a.pdf
    layer2_hoopstudent.json
  processed/
    layer1_rulebook_chunks.json
    layer2_chunks.json
  embeddings_checkpoint.json
```

4 Implementation Journey

Below is a detailed account of the full implementation journey for the first iteration (V1.0) of the system. Modifications made specifically for the ablation study are discussed in the following chapter.

4.1 Pre-processing

Pre-processing differs substantially between the two layers due to their source formats.

Layer 1: Rulebook PDF Processing

PDF processing for Layer 1 is handled by **Docling** (IBM Research). The choice of Docling over alternatives such as `pdfminer` or `PyMuPDF` was deliberate. Traditional PDF extraction libraries produce a flat text stream — headers, body content, and page numbers are interleaved without structural distinction. For a rulebook organised hierarchically into Rules and Sections, this means the extractor has no reliable way to identify where one rule ends and another begins without fragile regular expressions tuned specifically to that document's formatting.

Docling applies computer vision models to understand document layout, producing a **DoclingDocument** object with heading nodes already identified and placed in their correct hierarchical positions. This makes downstream processing significantly cleaner: the heading path for any chunk of text (e.g., *"RULE NO. 10 — Violations and Penalties → Section XIII — Traveling"*) is accessible directly from the chunk's metadata without any text parsing.

The Docling pipeline was configured to disable OCR and table structure analysis, since the rulebook text is native PDF text and no tables require extraction.

Heading Parsing

After Docling chunks the document, each chunk's heading path is parsed by a custom `parse_heading_path()` function using regular expressions designed to handle the formatting conventions of both the NBA and FIBA rulebooks. The function extracts:

- `rule_number`: e.g., *Rule 10*
- `rule_title`: e.g., *Violations And Penalties*
- `section`: e.g., *Section XIII*
- `section_title`: e.g., *Traveling*

- `chunk_type`: one of *rule_section*, *definitions*, *comments*, or *general*

This structured metadata is stored alongside each vector in Pinecone and is used by the generator to construct informative source labels in the UI.

Noise Filtering

A `is_noise_chunk()` filter removes chunks that have no retrieval value: chunks shorter than 80 characters, chunks consisting entirely of digits and dashes (page number artefacts), index entry patterns ("Rule No. X ... Page Y"), and chunks containing dense sequences of dot leaders (table-of-contents entries). This filter operates after chunking, on the raw text of each candidate chunk.

Context Prefix

Each chunk's text is prepended with a natural-language context prefix constructed from its heading metadata:

```
"NBA Rulebook | Rule 10 (Violations And Penalties),
 Section XIII (Traveling):\n\n{chunk_text}"
```

This prefix serves an important retrieval function: without it, a chunk about traveling merely reads "A player who receives the ball while standing still may pivot...". There is no explicit indication that the text is from the NBA rulebook, from Rule 10, or even about basketball. With the prefix, BM25's sparse retrieval leg can match on "Rule 10", "Traveling", "NBA", and "Violations" as explicit keyword signals, substantially improving hybrid search precision for rule-specific queries.

Layer 2: HoopStudent JSON Pre-processing

Layer 2 pre-processing (`preprocess_layer2.py`) converts the scraped JSON documents into the standardised chunk format expected by the embedding and indexing pipeline. Each term document produces two types of chunks:

1. **Definition chunk.** A single short chunk constructed from the `concise_definition` field, formatted as `''{term}: {definition}''`. This chunk is optimised for exact-match queries ("what is a pick and roll?") because the term name and its definition are co-located in a small, dense text window, improving both BM25 keyword matching and dense embedding precision.

2. **Section chunks.** One chunk per section from the detail page, formatted with a context prefix: `''{term} | {section_heading}:\n\n{section_text}''`. These chunks carry the detailed explanatory content and handle queries that require more than a one-line answer (e.g., "how does a floppy play work against a switching defence?").

Sections with headings in a defined skip-list ("Table of Contents", "Introduction", navigational boilerplate) and sections shorter than 80 characters are discarded.

4.2 Chunking

Layer 1: HybridChunker

For the rulebook PDFs, chunking is performed by Docling's **HybridChunker**, which performs Document Specific Chunking. The alternative, `HierarchicalChunker`, splits strictly at heading boundaries regardless of section size — a two-sentence section such as "Section V — Free Throw" becomes a standalone chunk too short to embed meaningfully.

`HybridChunker` applies a smarter strategy: it merges short adjacent sections that share the same parent heading, splits long sections at sentence boundaries when they exceed the token limit, and preserves the heading hierarchy throughout. For a corpus where section lengths range from two sentences to three pages, this results in a significantly more uniform chunk size distribution.

The token limit was set to **500 tokens** using the BGE-M3 tokenizer (`BAAI/bge-m3`). BGE-M3 supports a maximum sequence length of 8192 tokens, but shorter chunks embed more precisely — a chunk that spans two unrelated sections risks producing an embedding that represents neither adequately. Critically, the **same tokenizer** must be used for chunking as for embedding, to ensure token counts are accurate.

Layer 2: Fixed Schema Chunking

Layer 2 does not use a sliding window or a general-purpose chunker. The scraped documents have a natural, pre-existing structure (concise definition + discrete sections), so the chunking strategy follows that structure directly. This avoids the risk of a fixed-window chunker splitting a definition mid-sentence or merging two unrelated sections.

4.3 Embedding

Model Selection: BGE-M3

The embedding model is **BGE-M3** (`BAAI/bge-m3`). The original plan involved separate dense and sparse encoders: `all-MiniLM-L6-v2` for dense embeddings and Pinecone's `BM25Encoder` for sparse. This was replaced by BGE-M3 for the following reasons:

1. BGE embedding models are very popular for their quality as well as effeciency.
2. BGE-M3 generates both a 1024-dimensional dense vector and a learned sparse vector (lexical weights) in a **single forward pass**, with no separate encoder or vocabulary

fitting required. The sparse representation is qualitatively superior to classical BM25: rather than frequency-based IDF weights over a fixed vocabulary, BGE-M3's lexical weights are learned from training data and reflect the actual semantic importance of tokens in context. This produces better keyword matching, particularly for domain-specific terminology such as basketball rule identifiers.

Sparse Vector Format

BGE-M3's `lexical_weights` output is a dictionary mapping token identifiers to importance scores. This is converted to Pinecone's required sparse format ({`indices: [...], values: [...]`}) with two filtering steps: weights below 0.01 are discarded as noise, and the result is capped at the top 1000 tokens by weight. The 1000-token cap is a practical limit imposed by Pinecone's sparse vector constraints, not related to the 1024-dimensional dense vector.

Pinecone Index Architecture

A single hybrid Pinecone index is used with the following configuration:

- **Dimension:** 1024 (BGE-M3 dense output)
- **Metric:** `dotproduct` — mandatory for hybrid sparse-dense search in Pinecone

Each Pinecone record stores the dense vector, the sparse vector, and a metadata dict containing all fields from the chunk's `metadata` key, including the chunk text itself (stored under the `text` key for retrieval-time access without a separate document store).

Due to the scale of the embedding operation (several hundred chunks, BGE-M3's computational requirements), the embedding step was run on a Kaggle notebook with dual T4 GPUs, as mentioned above. Due to network failures during Pinecone upsert, re-running the full embedding pass was required. To avoid this, checkpointing was used.

4.4 Retrieval

The retrieval pipeline (`retriever.py`) implements a four-stage process designed to balance recall, diversity, and precision:

1. **Query Encoding.** The query is encoded with BGE-M3 (same model used during indexing) to produce a 1024-dim dense vector and a learned sparse vector in one pass.
2. **Dual Pinecone Query:** Two separate Pinecone queries are issued against the same index: a pure dense query (`alpha = 1.0`) retrieving the top 20 results by semantic similarity, and a pure sparse query (`alpha = 0.0`) retrieving the top 20 results by

lexical weight overlap. These were run as separate queries, since we need two lists, a requirement for Reciprocal Rank Fusion.

3. **Reciprocal Rank Fusion (RRF):** The two ranked lists are fused using RRF with smoothing constant $k = 60$.

4. **CrossEncoder Re-ranking:** The top 10 candidates from RRF are passed to a CrossEncoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) for precision re-ranking. The CrossEncoder jointly encodes the query and each candidate as a single input, producing a relevance score. Applying it only to the small post-RRF candidate set (rather than the full corpus) keeps latency acceptable. The final top 5 candidates by CrossEncoder score are returned.

5. **Singleton loading:** Both BGE-M3 and the CrossEncoder are loaded once at application startup via singleton instances and reused across all requests.

A note on MMR. Maximal Marginal Relevance (MMR) was implemented in the pipeline but was disabled in V1.0, since I wanted to see if it would be required or not. (the top-10 RRF results are passed directly to the CrossEncoder without diversity re-selection).

4.5 Generation

Model Selection Journey

Selecting a reliable, free generation backend proved to be one of the more operationally challenging aspects of the project. The intended model was `mistralai/Mistral-7B-Instruct-v0.3` via the HuggingFace Inference API. The journey through providers is summarised below:

- **HuggingFace Inference API (hf-inference):** No longer supports large LLM chat completion at the free tier.
- **Novita:** Returned 404 for Mistral.
- **Cerebras:** The older `huggingface-hub` library used incompatible chat completion syntax; resolved by switching to `.chat.completions.create()`, but the provider itself had reliability issues.
- **SambaNova:** Functional but drained the monthly HuggingFace credit allocation.
- **OpenRouter + `google/gemma-3-12b-it:free`:** Permanently free, OpenAI API-compatible (requiring no library changes beyond `base_url`), and backed by multiple providers for uptime. However, it was limited to 50 requests per day, making testing difficult.

- **Groq** + `llama-3.1-8b-instant:` Groq proved to be the most friendly API, with flexible rate limits applied per minute. Llama3.1-8B was also a good model according to online research for RAG based applications. This was finalized and used for all the implementations.

Prompt Design

The prompt enforces three faithfulness behaviours explicitly:

1. **Context-only answering.** The model is instructed to answer *only* from the numbered `[Source N]` blocks in the prompt and not to draw on parametric knowledge.
2. **Inline citation.** The model must cite sources as `[Source N]` inline, enabling the UI to show the corresponding retrieved chunks.
3. **Explicit uncertainty.** If the retrieved context does not contain enough information to answer, the model is instructed to say so explicitly rather than speculate. This directly improves the faithfulness score in evaluation.

Each retrieved chunk is formatted in the context block as:

```
[Source N | {League} Rulebook | {Section}]
{chunk_text}
```

or, for Layer 2 chunks:

```
[Source N | HoopStudent Basketball Encyclopedia | {Term} | {Section Type}]
{chunk_text}
```

One Gemma-specific issue was encountered: Gemma's chat template does not support the `system` role. The system prompt was therefore merged into the `user` turn.

4.6 Evaluation

The evaluation framework (`evaluate.py`) implements LLM-as-a-Judge scoring across two metrics. I specifically did not go for the RAGAS methodology since they have a black-box design, and preferred the custom approach for learning.

Faithfulness

Faithfulness measures the fraction of factual claims in the generated answer that are directly supported by the retrieved context — independent of the claim's factual truth in the real world. The pipeline proceeds in two steps:

Step 1 — Claim Extraction. The judge LLM is prompted to extract every atomic factual claim from the generated answer and return them as a JSON array. Atomic claims are short declarative sentences asserting a single fact; hedges, meta-statements, and opinions are excluded. The JSON output format makes parsing deterministic and avoids line-based fallbacks in the common case.

Step 2 — Claim Verification. Each extracted claim is presented to the judge alongside the full concatenated retrieved context. The judge returns a structured verdict in the format `VERDICT: SUPPORTED / NOT SUPPORTED` followed by a one-sentence reason. The regex parse targets the `VERDICT:` prefix specifically, making extraction robust to variation in the surrounding prose.

The faithfulness score is the fraction of supported claims

Answer Relevancy

Relevancy measures how well the generated answer addresses the original query, using the following steps:

Step 1 — Question Generation. The judge LLM generates three distinct questions that the generated answer could plausibly be a response to.

Step 2 — Embedding Similarity. All three generated questions and the original query are embedded using `sentence-transformers/all-MiniLM-L6-v2`, a lightweight 80 MB model that runs efficiently on CPU. Since embeddings are L2-normalised, cosine similarity reduces to a dot product. The relevancy score is the mean cosine similarity between each generated question and the original query:

$$\text{Relevancy} = \frac{1}{3} \sum_{i=1}^{3} \cos(\mathbf{q}_{\text{orig}},\, \mathbf{q}_i)$$

The use of `all-MiniLM-L6-v2` for relevancy (rather than BGE-M3) is intentional: the similarity computation is a lightweight sentence-pair comparison for which a small bi-encoder is entirely adequate, and avoiding a second BGE-M3 load keeps memory pressure manageable on the free-tier deployment.

Judge Model and Reproducibility

The idea with the judge model was to use a smaller LLM whose only task would be for claim extraction, claim verification and question generation aspect of the evaluation.

All judge LLM calls use `temperature = 0.0` to make scores reproducible across evaluation runs. The judge client was initially configured to use OpenRouter (same as the generator), but was subsequently switched to Cohere's `tiny-aya` model due to OpenRouter's daily request limits being insufficient for a 10-query evaluation set with multi-step per-query calls)

(more detail on this later). The structured output format (`VERDICT:` prefix, JSON arrays) was designed to be parser-friendly regardless of which judge model is in use.

Test Set

A fixed set of 10 basketball queries is used for evaluation, spanning three categories: NBA-specific rules (defensive three-second violation, out-of-bounds procedures, technical fouls, shot clock reset rules), FIBA-specific rules (shot clock duration, backcourt violations, in-bound timing), and tactical strategy (zone vs. man-to-man defense, box-and-one purpose, isolation plays). Using a fixed test set ensures that ablation study results are directly comparable across configurations.

(the test queries are available on github)

UI Integration

Evaluation results are streamed to the front-end via a Server-Sent Events (SSE) endpoint so that relevancy and faithfulness scores appear progressively as each query completes, rather than after the full 15-query batch. For faithfulness, the reasoning of the LLM for declaring the claim as Supported or Unsupported is also provided.

For automated evaluation of the test set, there is a "Evaluate All" button.

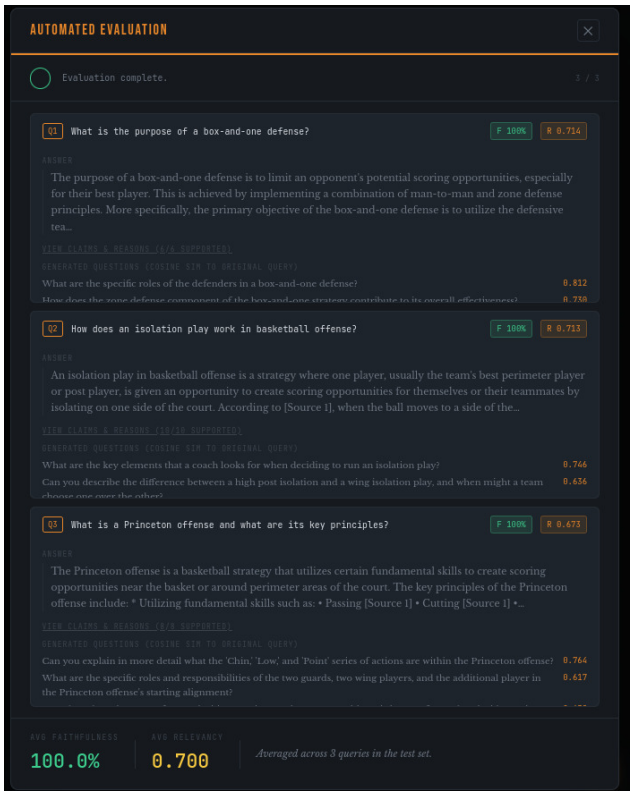

Figure 4.1: How it looks in the UI, on a reduced test set

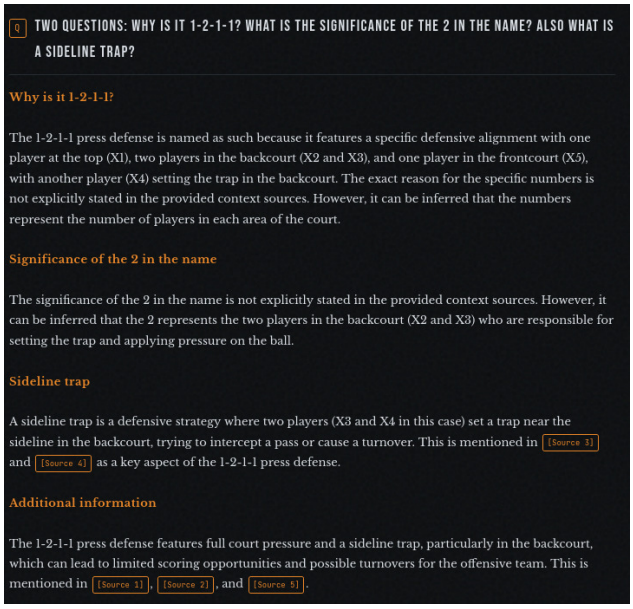

Figure 4.2: V1.0 output

NOTE: the UI works best on laptops/PCs, isn't mobile friendly since that was not the scope of the project.

There is also a latency in the first prompt since that actually loads the retriever, generator, evaluator model/objects, a restriction of HuggingFace in their free tier

Some example output:

5 Improvements to the First Iteration

After completing the end-to-end pipeline described in Chapter 4, several issues were identified through both automated evaluation and hands-on human testing. These issues were addressed in two incremental releases, **V1.1** and **V1.2**, before the system was extended with alternative chunking strategies for the ablation study. The source code for each version is preserved on its own branch in the GitHub repository; the `README` on each branch summarises the changes relative to the base implementation.

5.1 V1.1 — Evaluation LLM Swap and Prompt Rework

Challenge 1: Judge LLM Rate Limiting

The original judge LLM was routed through OpenRouter. A 15-query evaluation run involves multiple multi-step API calls per query (claim extraction, then per-claim verification, then question generation for relevancy), which rapidly exhausted OpenRouter's daily request ceiling of 50 calls.

Fix: The judge client was switched to the **Gemma 3.1 4B** model served via a separate provider with more permissive per-minute rate limits. To prevent the automated "Evaluate All" run from hitting per-minute caps on consecutive queries, a **60-second inter-query sleep** was inserted in `evaluate.py` between each of the 15 test cases. This adds roughly 14 minutes to a full automated evaluation pass but ensures that no request is dropped due to rate limiting. The structured output format (`VERDICT:` prefix and JSON arrays) required only minor prompt adjustments to remain compatible with Gemma's output style.

A note on Pinecone namespaces: the initial HybridChunker (V1.2) implementation used the **default (empty string) namespace** in Pinecone, sinceI discovered namespaces as a feature only after the first implementation was complete. All subsequent chunking strategies — fixed-size and semantic — were upserted into their own named namespaces so that the same Pinecone index could host all three corpus variants simultaneously and be queried independently.

Challenge 2: Generation Prompt Producing Unformatted Walls of Text

With the initial prompt instructing the model to be "concise but complete" while strictly citing sources, the LLM responded by mechanically stitching factual sentences together without any structural formatting. The resulting answers were accurate but extremely difficult to read.

Fix: The generation system prompt was redesigned following the instruction-hierarchy principle from prompt engineering best practices. The revised prompt explicitly asks the model to

structure its response with a brief introduction, organised body paragraphs (using markdown-style headers for multi-part answers), and a concise conclusion. Inline citations (`[Source N]`) are still mandatory, but the model is now additionally instructed to *paraphrase* retrieved content rather than transcribe it verbatim. The evaluation prompt was similarly relaxed to allow for faithfulness judgment to be based on semantic equivalence rather than lexical overlap, preventing correct paraphrases from being scored as unsupported.

Challenge 3: Dependency Environment Conflict (No Fix)

A version conflict was discovered between the `transformers` library required by **Docling** for PDF layout parsing and the version required by **FlagEmbedding** (BGE-M3) for embedding. The two packages cannot coexist in the same Python environment without one overwriting the other's `transformers` pin.

Workaround: Pre-processing (Docling) and embedding (BGE-M3) were run in *separate Kaggle notebook environments* on separate kernels. The chunked output from the pre-processing notebook was saved as a JSON checkpoint file and loaded directly into the embedding notebook, sidestepping the conflict entirely at the cost of a manual handoff step.

5.2 V1.2 — Diagnosing the Comparative Query Failure

After deploying V1.1, a critical retrieval failure was uncovered through **human evaluation**: for the query

> *"Coach's challenge: we have a very athletic team but we are undersized. Which press defense [1-2-1-1 or 1-2-2] should we use to maximise our speed for turnovers?"*

the system consistently retrieved sources covering only the **1-2-1-1** press and largely ignored the **1-2-2**. The generated answer therefore could not make a meaningful comparison, even though both defences are present in the corpus. Crucially, this failure was *not* visible in the automated faithfulness or relevancy scores — the answer scored 83% faithfulness and a relevancy of 0.68, both values that would not flag a problem. This illustrates a key limitation of purely automated evaluation: metrics that assess grounding and answer-question alignment cannot detect *retrieval coverage gaps* that manifest only through human reading of the response.

Figure 5.1: The response on the UI

Why Hybrid Retrieval Fails on Comparative Queries

The failure is a structural property of single-vector retrieval. When the query mentions both defences, BGE-M3 encodes it into **one** dense vector representing the "average" semantic meaning of the entire query. Because 1-2-1-1 appears more prominently in the query text (and has significantly more chunks in the index since it is more widely documented), the resulting dense vector is pulled toward the 1-2-1-1 region of the embedding space. The retrieval pipeline then surfaces 15–20 1-2-1-1 chunks from the dense top-20, with the 1-2-2 chunks ranked low enough that they fall outside the top-20 candidate pool entirely.

Why MMR Did Not Fully Resolve This

Maximal Marginal Relevance (MMR) was activated at $\lambda = 0.7$ (a value selected by experimentation, balancing relevance and diversity — $\lambda = 0.6$ introduced too much randomness on specific queries while $\lambda = 0.9$ produced no visible diversity benefit) in an attempt to force a more varied candidate set. MMR did surface a **2-1-2** press chunk that had not appeared in the baseline response, but the **1-2-2** chunk remained absent.

Figure 5.2: Notice how LLM talks about the 2-1-2 chunk with MMR

The reason is a bottleneck in the RRF pool. The MMR step operates on only the **top 15** candidates from RRF fusion. If 1-2-2 chunks are ranked 16th–20th in RRF (i.e., they appear in the sparse top-20 but not in the dense top-20), they are eliminated before MMR even sees them. The 2-1-2 press appeared because it happened to rank just inside the top-15 cut; the 1-2-2 did not. MMR cannot add diversity to candidates that were never admitted to its input pool.

This limitation is fundamentally architectural: a pipeline that encodes the full query into a single embedding vector and retrieves a single ranked list cannot guarantee coverage of all named entities in a multi-entity comparative query. A proper solution would require query decomposition (encoding each entity as a separate sub-query) or multi-vector representations, both of which are out of scope for the current system.

Observation: Semantic Chunking Resolved This Naturally

Notably, the same comparative query was answered correctly when the corpus was indexed under **semantic chunking**. The likely explanation is that semantic chunking's percentile-breakpoint algorithm groups sentences by topic shift rather than by token budget, producing chunks that are more densely focused on a single defensive concept. A 1-2-2-specific chunk therefore lands closer to the 1-2-2 region of the embedding space than it would under fixed-window or document-structure-based chunking, making it more recoverable by the dense retriever even when the query vector is pulled toward 1-2-1-1. This observation is one of the strongest qualitative arguments for semantic chunking in this corpus and reinforces the ablation study results discussed in Chapter 6.

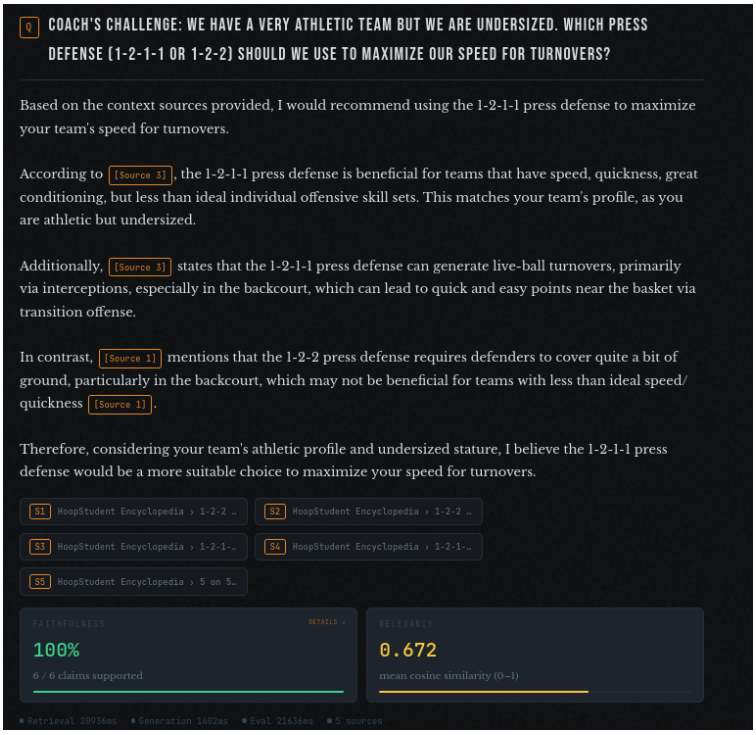

Figure 5.3: Compare this response from the semantic chunking version to V1.2's

6 Alternative Chunking Strategies and Ablation Study

To compare the effects of different chunking strategies and retrieval modes on system performance, two additional chunking approaches were implemented and evaluated against the baseline Docling HybridChunker (V1.2). All strategies were evaluated on a fixed test set of **10 queries** spanning the same three categories used in the full 15-query evaluation (NBA rules, FIBA rules, and tactical strategy), using the LLM-as-a-Judge framework described in Chapter 4. The "Evaluate All" button in the deployed UI runs this automated test set. Per-query results for each configuration are presented in the subsections below; all averages are rounded to two decimal places.

6.1 Fixed-Size Chunking

Fixed-size chunking retains Docling for text *extraction and noise filtering* (since its computer vision models are still required for reliable text recovery from the rulebook PDFs) but discards Docling's document-structure-aware chunk boundaries in favour of a **sliding window** with a fixed token limit and token overlap. The resulting chunks are semantically agnostic: boundaries are placed wherever the token budget is exhausted, regardless of sentence or section boundaries.

- **Layer 1 chunks produced:** 245
- **Layer 2 chunks produced:** 3,006
- **Chunk token limit:** 512 tokens with overlap

A notable operational consequence of fixed-size chunking is increased latency. Full-length 512-token chunks are substantially larger than the typical HybridChunker output, which means each Pinecone record carries a larger metadata payload and each CrossEncoder inference takes longer. More significantly, the larger chunk sizes cause the judge LLM's per-claim verification requests to approach the API provider's per-minute token budget more quickly, resulting in frequent rate-limit throttling during the evaluation run.

Rationale for Testing Hybrid-Only

Fixed-size chunking was evaluated under the **hybrid retrieval (RRF + MMR)** configuration only, and not extended to dense-only or sparse-only ablations. The hybrid run already produced the **worst mean faithfulness score** of all configurations tested (82.8%), making it clear that fixed-size chunking is not a competitive strategy for this corpus. Running additional retrieval mode ablations on an already-underperforming chunking strategy would

yield no actionable insight and would consume significant evaluation API budget. The data is therefore intentionally limited to a single configuration.

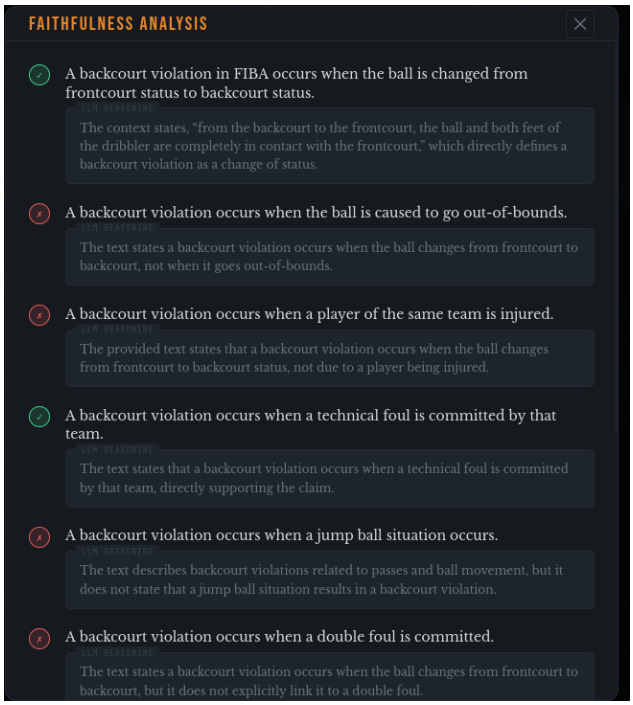

Figure 6.1: This is the evaluation of a response to the 5th query in the test set with fixed-size chunk strategy

Per-Query Results

Table 6.1: Fixed-Size Chunking — Hybrid Retrieval (RRF + MMR). Per-query evaluation results.

Query #	Faithfulness (%)	Relevancy Score
1	100.0	0.737
2	100.0	0.662
3	95.0	0.678
4	100.0	0.768
5	44.0	0.775
6	50.0	0.621
7	80.0	0.731
8	90.0	0.709
9	100.0	0.748
10	69.0	0.604
Average	**82.8**	**0.703**

Analysis

The low faithfulness scores on queries 5 (44%) and 6 (50%) are the primary drivers of the poor average. These outliers are characteristic of fixed-size chunking's core weakness: a rule description or tactical explanation that spans a natural section boundary is split mid-concept. The retriever surfaces one half of the explanation, and the generator either hallucinate-fills the gap or states uncertainty — both outcomes reduce faithfulness. The moderate relevancy scores (average 0.703) suggest that the retrieved chunks are topically adjacent to the queries but lack the precise conceptual grounding needed for fully supported answers.

6.2 Semantic Chunking

Description

Semantic chunking segments the text by *topic shift* rather than by token budget or document structure. The implementation follows the Greg Kamradt / LlamaIndex percentile-breakpoint approach:

1. Sentence-tokenise each text block using NLTK's `sent_tokenize`.
2. Build context windows: each window is sentence i concatenated with sentence $i{+}1$, giving the encoder a lookahead over what follows.
3. Embed every context window with `all-MiniLM-L6-v2` (already resident in memory from `evaluate.py`, so no extra model load).
4. Compute cosine distance between consecutive window embeddings.
5. Place a chunk boundary wherever the distance exceeds the **85th percentile** of all pairwise distances — these are the points at which topic demonstrably shifts.
6. Aggregate sentences between consecutive boundaries into one chunk.
7. Hard-split any chunk exceeding ∼2,000 characters (∼500 tokens) at a sentence boundary as a fallback to prevent overlength vectors.
8. Drop chunks shorter than 80 characters as noise.

The 85th percentile threshold is configurable via `BREAKPOINT_PERCENTILE`. Raising it toward 95 yields fewer, larger chunks; lowering it toward 70 yields many smaller ones.

- **Layer 1 chunks produced:** 682
- **Layer 2 chunks produced:** 5,865

- **Chunking time:** Longest of all three strategies due to per-sentence embedding of the entire corpus.

Three retrieval configurations were evaluated: **hybrid** (RRF + MMR, λ=0.7), **dense-only** (alpha = 1.0, no sparse component), and **sparse-only** (alpha = 0.0, no dense component).

Per-Query Results — Hybrid (RRF + MMR)

Table 6.2: Semantic Chunking — Hybrid Retrieval (RRF + MMR, λ=0.7). Per-query results.

Query #	Faithfulness (%)	Relevancy Score
1	100.0	0.624
2	100.0	0.686
3	100.0	0.633
4	100.0	0.784
5	100.0	0.704
6	100.0	0.724
7	83.3	0.674
8	85.7	0.681
9	100.0	0.764
10	100.0	0.663
Average	**96.9**	**0.694**

Per-Query Results — Dense-Only

Table 6.3: Semantic Chunking — Dense-Only Retrieval (alpha = 1.0). Per-query results.

Query #	Faithfulness (%)	Relevancy Score
1	100.0	0.724
2	100.0	0.713
3	93.0	0.679
4	100.0	0.836
5	100.0	0.823
6	100.0	0.718
7	100.0	0.689
8	90.0	0.870
9	100.0	0.813
10	100.0	0.736
Average	**98.3**	**0.760**

Per-Query Results — Sparse-Only

Table 6.4: Semantic Chunking — Sparse-Only Retrieval (alpha = 0.0). Per-query results.

Query #	**Faithfulness (%)**	**Relevancy Score**
1	100.0	0.707
2	100.0	0.675
3	94.0	0.691
4	100.0	0.791
5	100.0	0.797
6	75.0	0.776
7	100.0	0.698
8	100.0	0.781
9	100.0	0.707
10	100.0	0.688
Average	**96.9**	**0.731**

Analysis

Semantic chunking consistently outperforms fixed-size chunking across all retrieval modes. The high faithfulness scores reflect the fact that topic-coherent chunks give the generator a focused, unambiguous context window — the LLM is less likely to encounter a half-explained concept mid-chunk and therefore less likely to fill the gap with unsupported inference.

An interesting and somewhat counterintuitive result is that **dense-only retrieval produced the highest scores**, both in faithfulness (98.3%) and relevancy (0.760), outperforming the hybrid configuration. The hybrid run's relevancy is notably lower (0.694). This can be attributed to the following: with semantically coherent chunks already tightly focused on individual basketball concepts, the BGE-M3 dense vector alone captures the query intent with high precision. Adding the sparse (lexical) leg introduces some noise — keyword-heavy chunks that are lexically close to the query but not the most semantically relevant — which pulls the final top-5 slightly off-target. This suggests that for a well-organised, semantically structured corpus, dense-only retrieval may be the preferred mode, with hybrid retrieval reserved for rule-specific queries that require exact keyword matching (e.g., "Rule 10, Section XIII").

The sparse-only configuration achieved the same faithfulness as hybrid (96.9%) but a higher relevancy (0.731 vs. 0.694), placing it between the two. This confirms that BGE-M3's learned lexical weights are a strong standalone retriever on this corpus, and that the primary value-add of the dense leg is in semantic generalisation across paraphrased queries.

6.3 V1.2 — Docling HybridChunker (Baseline)

Description

V1.2 is the final strategy reached from the implementation journey explained above. Docling's `HybridChunker` determines chunk boundaries by merging short adjacent sections sharing the same parent heading, splitting overlength sections at sentence boundaries, and preserving the full heading hierarchy throughout. Two retrieval ablations were run: **hybrid (RRF + MMR)** and **dense-only**. A sparse-only run was not conducted as the hybrid run already revealed a structural problem with the corpus representation under this chunking strategy (the comparative query failure discussed in Chapter 5).

- **Layer 1 + Layer 2 chunks produced:** 5496

Per-Query Results — Hybrid (RRF + MMR)

Table 6.5: V1.2 (Docling HybridChunker) — Hybrid Retrieval (RRF + MMR). Per-query results.

Query #	Faithfulness (%)	Relevancy Score
1	100.0	0.550
2	100.0	0.563
3	81.0	0.731
4	100.0	0.830
5	100.0	0.820
6	100.0	0.740
7	83.0	0.676
8	100.0	0.770
9	100.0	0.788
10	75.0	0.700
Average	**86.5**	**0.717**

Note: Query 7 scored 83% faithfulness with relevancy 0.676. Human inspection confirmed that this query was incorrectly answered because the retriever did not surface the 1-2-2 press chunk — the comparative query failure discussed in Chapter 5 (Section 5.2).

Per-Query Results — Dense-Only

Table 6.6: V1.2 (Docling HybridChunker) — Dense-Only Retrieval. Per-query results.

Query #	**Faithfulness (%)**	**Relevancy Score**
1	100.0	0.786
2	100.0	0.630
3	85.0	0.760
4	100.0	0.804
5	100.0	0.644
6	100.0	0.792
7	86.0	0.675
8	89.0	0.793
9	100.0	0.755
10	90.0	0.707
Average	**95.0**	**0.735**

Analysis

The dense-only configuration substantially outperforms the hybrid for V1.2, raising faithfulness from 86.5% to 95.0%. The improvement confirms that adding the sparse retrieval leg *under HybridChunker* introduces more noise than signal for this corpus: the sparse leg is effective at matching rule identifiers, but HybridChunker chunks — which already carry explicit rule-number prefixes in their context strings — provide sufficient keyword signal for the dense model to locate them without explicit BM25-style matching. The sparse leg's contribution is therefore partially redundant while still admitting keyword-adjacent but semantically off-target chunks into the candidate pool.

6.4 Ablation Study: Summary and Discussion

Table 6.7 presents the aggregated results across all evaluated configurations.

Table 6.7: Ablation Study Summary — Average Faithfulness and Relevancy across all configurations. RRF = Reciprocal Rank Fusion; MMR = Maximal Marginal Relevance ($\lambda = 0.7$). "—" indicates that the configuration was not evaluated.

Chunking Strategy	**Retrieval Mode**	**RRF**	**MMR**	**Avg. Faith. (%)**	**Avg. Rel.**
Semantic Chunking	Hybrid	✓	✓	96.9	0.694
	Dense Only	—	—	**98.3**	**0.760**
	Sparse Only	—	—	96.9	0.731
V1.2 (Docling HybridChunker)	Hybrid	✓	✓	86.5	0.717
	Dense Only	—	—	95.0	0.735
Fixed-Size Chunking	Hybrid	✓	✓	82.8	0.703

Key Findings

1. Semantic chunking is the strongest overall strategy. Across all retrieval modes, semantic chunking produces the highest faithfulness and relevancy scores. Its topic-shift-based boundaries prevent the concept-splitting artefacts that cause faithfulness failures under the other strategies, and the resulting dense chunks embed more precisely in the BGE-M3 space, improving both dense and hybrid retrieval precision. The ability to correctly handle the comparative press-defence query, which both V1.2 and fixed-size chunking failed, further reinforces this finding.

2. Dense-only retrieval consistently outperforms hybrid for this corpus. Across both semantic chunking and V1.2, removing the sparse retrieval leg *improves* performance. This is counterintuitive given that hybrid retrieval is widely recommended for technical corpora with domain-specific identifiers. **The explanation is that this corpus has been pre-enriched with explicit keyword context prefixes (rule numbers, league names, term names) embedded directly in each chunk's text, which means the dense embedding already captures strong lexical signal**. The sparse leg's additional keyword matching therefore provides diminishing returns while introducing noise from lexically proximate but semantically irrelevant chunks.

3. Fixed-size chunking is the weakest strategy. The 82.8% faithfulness under hybrid retrieval is the lowest of all configurations, and the two low-scoring outlier queries (44% and 50%) confirm the expected failure mode: fixed windows split explanations mid-concept, leaving the generator without the full context needed to ground its claims.

4. Sparse-only retrieval is a viable baseline under semantic chunking. With 96.9% faithfulness and a relevancy of 0.731 — higher relevancy than hybrid — the sparse-only configuration performs on par with hybrid while being computationally cheaper (no dense encoding at retrieval time). This suggests that BGE-M3's learned lexical weights are a strong standalone retriever when chunks are topic-coherent, and that the value of the dense leg lies primarily in handling queries that are semantically related to a concept but do not share its

vocabulary.

5. Human evaluation revealed a failure mode invisible to automated metrics. The comparative query failure (Chapter 5.2) scored 83% faithfulness and 0.676 relevancy under automated evaluation — values that would not trigger concern in a metrics-only review. This underscores the necessity of complementing RAGAS-style automated evaluation with at least a small set of manually constructed adversarial queries, particularly multi-entity comparison queries, which are structurally the hardest case for single-vector RAG systems.

7 Conclusion

The winning configuration — **semantic chunking with dense-only retrieval** is the version currently live at the HuggingFace page.

One regret is to not be able to incorporate the WNBA and NCAA rulebooks, for more comprehensive rule comparison, as well as not being able to access NBA team-specific playbooks for the layer 2. I wish to work on in in the future

Also. future work could address the comparative query limitation, extend the corpus to include game film transcripts or coaching manuals, or incorporate visual models to understand screenshots of plays.